Computer Graphics: Lecture 15

Lighting part 1: Diffuse (Lambertian) Lighting

Welcome back!

Last time we learned about real, hierarchical scene management.

We made a simple recursive scene-based renderer.

This was important: recursive scene nodes are the core of most 3D graphics engines.

There's something we're still missing though, and it's very important for realism in our scenes…

...Lights!

Lighting is critical for human perception of an object.

Without it, the only other real option is texturing, but it's just not enough on its own.

In this lecture, we're going to take steps to implement the lighting system you see in this screenshot.

How does light work?

We probably learned the basics in science class:

- Light is emitted as photons from light sources, such as the sun.
- The photons get reflected, refracted, or absorbed as they move.
- If one of them happens to bounce into our eye, there's a chance one of our eye's rods or cones will pick it up, and aggregate it with the other photons happen to be hitting at around that time.
- Then we *see* the light.

So, seems to suggest a simple algorithm: track the photons as they leave the light source. [Why not?]

The problem

There are a *lot* of photons, and most of them don't hit our eyes.

As a result, simulating those would be entirely wasted time for the purpose of computer graphics.

However, what if we went in reverse?

That is, instead of tracking each lightbeam as it bounces around, what if we shot beams the other direction, out of our camera, and then checked if they hit a light source?

That works

That algorithm works; it's called **raytracing**.

Raytracing fundamentally works like this:

- For each pixel in the screen, fire a ray out into the world
- If it intersects a triangle, determine which light-sources are visible from the point of intersection.
- Based on the distances to those light-sources, we can add the color contributions from each one, and set the color of the pixel.

That's it. It's actualy a very simple algorithm. In fact, people have made raytracers that fit on the back of a business card.

So why aren't we making raytracers all the time?

Raytracing is a great algorithm. It's elegant, and it has the capacity to produce photo-realistic graphics. Unfortunately, it's still pretty slow.

There's a reason why RTX was such a big deal: real-time raytracing was kind of the holy grail of the computer graphics space for a while.

Even now, it's still rather limited in its effects.

What's so slow about raytracing?

The issue with raytracing

There are two dominant approaches to 3D graphics:

- Rasterization (what we're doing)
- Raytracing

Rasterization means "take this list of triangles, and draw their pixels to the screen."

Raytracing means "take this list of pixels and draw them according to which triangles they intersect".

The algorithms seem similar. However, modern renderers, even if RTX is supported, will do most of their drawing work with rasterization.

The issue with raytracing (2)

The issue with raytracing is that *testing whether a ray collided with one of the millions of triangles in your scene is slow.*

Let's do some basic math. If you're in 2160p resolution with 16:9 aspect ratio, sometimes known misleadingly as 4K, there will be 3840 by 2160 pixels. Roughly 8 million.

Each of those needs to be tested somehow against the triangles in the scene, which can also number in the millions.

Yes, there are all kinds of optimizations we can use to reduce the number of intersection tests, but the fact remains that the problem is hard.

The issue with raytracing (3)

When we draw a triangle with rasterization, we don't have to check whether it was collided with, we just draw the triangle.

Basically, "here are the bounds of the triangle, fill it with the pixel colors that get returned from the fragment shader."

There are still a large number of pixels, to fill in, but if a triangle is off screen, it will get clipped and we won't fill its pixels in.

As a result, instead of being in the order of number-of-pixels times number-of-triangles, it ends up being in the order of number-of-pixels plus number-of-triangles (assuming a non-degenerate case, like drawing a million triangles in order from far to near)

So why bother?

And yet, RTX seems to be here to stay, and raytracing doesn't seem to be going anywhere. So why? What's the big deal?

The main reason is this: raytracing enables global illumination without requiring "hacks".[1]

What is global illumination?

[1]There are other techniques besides raytracing that enable global illumination, such as Radiosity.

Global illumination

The lighting technique that we're going to learn today is a **local illumination** technique.

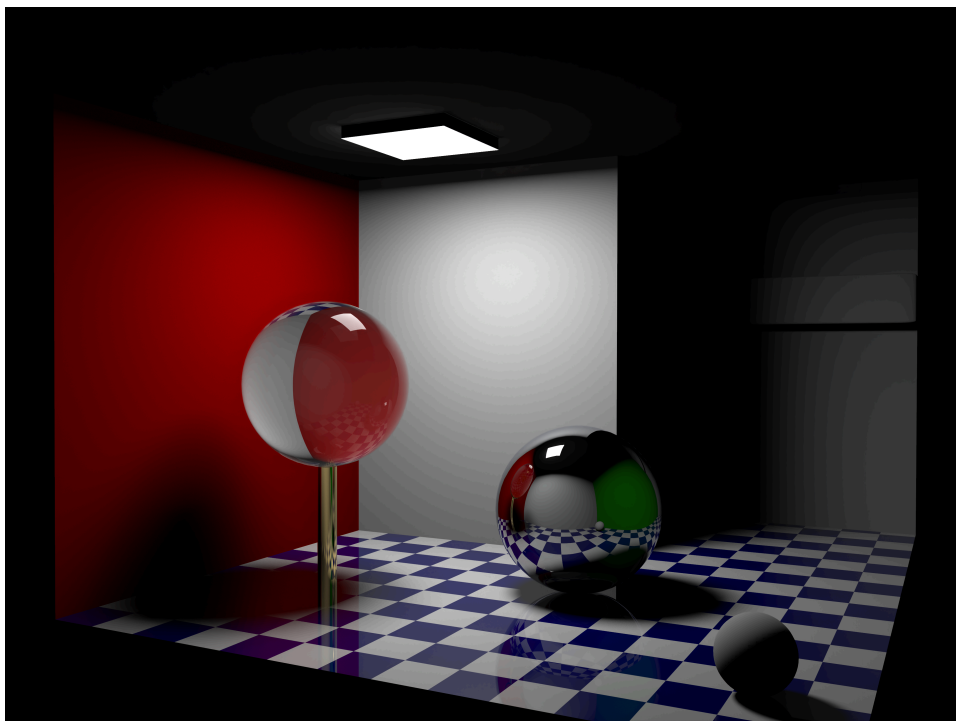
It works to help define what objects are in the scene and provide some basic realism, but it will never be perceived as photo-realistic.

The main issue is that we will *only be considering the impact of lights on the colors of our surfaces.*

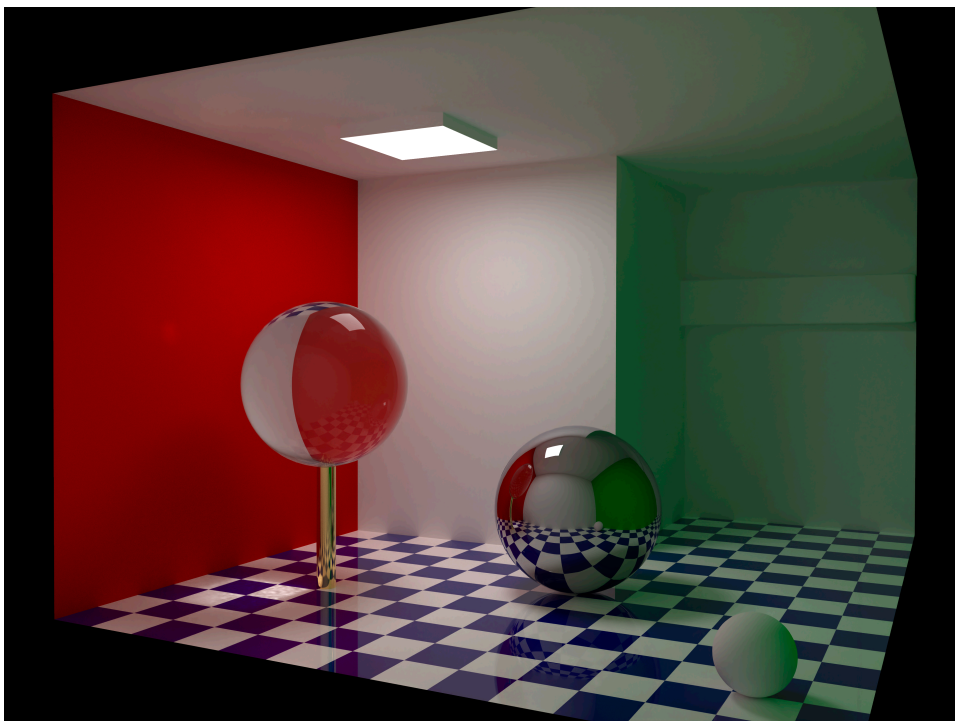
Only? What's the other option?

Well, when a light is cast upon a surface, some of it will be reflected. *The surface ends up becoming a light source.*

Global illumination vs local illumination



Local illumination only



Global illumination

Things to notice about the comparison

In both images, the light at the top is the primary source of light.

In the left image, that is the *only* source of light. As a result, it looks dark. The back wall is in shadow; it's not in direct view of the light.

In the right image, light is conserved. The walls end up being their own light source. It affects how we draw the objects: notice that the ball is green-ish on its right side. From that, we can infer that there is a green wall to the right that we can't see. We also see the green color cast on the white wall in the back, confirming this.

Global illumination is required for photorealism. It's also very slow.

Why global illumination is slow

Think back to our discussion on raytracing: where an “anti-photon” is sent out from the camera to see which triangle it hits.

We said that when an intersection is detected, we then check to see how illuminated that point is by a light source.

In global illumination, *everything is a light source.*

As a result, we need to send out more rays to see what *those* hit. And those raycasts could potentially require more rays. It can become an exponential problem if we don’t set a threshold for how much light we care about or how many ray-tests we will allow.

What to do instead

Even if we were to do global illumination, we still want to rasterize as much as we can get away with.

Light is additive, so if we draw a triangle with a good *base color*, we can add highlights onto it later by testing out some rays from the surface.

Ultimately, it becomes a game of identifying how to account for *most* of the light. If a triangle is directly under 4 light sources, we can probably sum those up and call it a day. It's only when the triangle is off in the corner and mainly illuminated by reflected light that we see a big benefit from ray tracing.

Starting from rasterization

This discussion has all been rather theoretical, as WebGPU doesn't support RTX or AMD's ProRender natively.

And you're definitely not getting realtime performance without hardware acceleration of ray intersection testing.

I just wanted you to understand the space, and know the difference between global and local illumination.

With that, let's take quesitons and start learning a much faster, local-illumination system...

Questions?

Basic lighting

There is an equation that fully describes the amount of light leaving a point in space along a viewing direction. It is called the rendering equation. Here it is:

$$L_{o(x,\omega_o,\lambda,t)} = L_{e(x,w_o,\lambda,t)} + L_{r(x,\omega_o,\lambda,t)}$$

$$L_{r(x,\omega_o,\lambda,t)} = \int_{\Omega} f_{r(x,\omega_i,\omega_o,\lambda,t)} L_{i(x,\omega_i,\lambda,t)} (\omega_i \cdot n) dw_i$$

So just plug and chug right?

Of course not. Most of those terms are only relevant under global-illumination. We can use a much simpler model.

Lambertian reflectance

The model we will learn in this lecture is called Lambertian Reflectance.

It is named after Johann Heinrich Lambert, and described in his book Photometria. The book contained, among other things, a bunch of experiments that were designed to learn how much light gets reflected in a variety of configurations.

This is before photography, so the book has visual setup guides, such as this one, depicting candles being placed in a correct configuration.

Despite the complicated setup, the book presents a remarkably simple rule for determining how much light is scattered from a surface, which we will learn soon.

Reflectance

There are several ways that light can interact with a surface:

- It can be absorbed by it.
- It can be "bent" (refracted)
- It can be reflected in a straight line. This is what happens to surfaces that have relatively few imperfections. This is **specular** reflection.
- It can be scattered in all directions. This is **diffuse** reflection.

The more chaotic a surface's microscopic texture, the more it will scatter and absorb light, and the less it will be reflected with a clear highlight.

Diffuse light

A surface that scatters light perfectly randomly in all directions is called a **lambertian** surface.

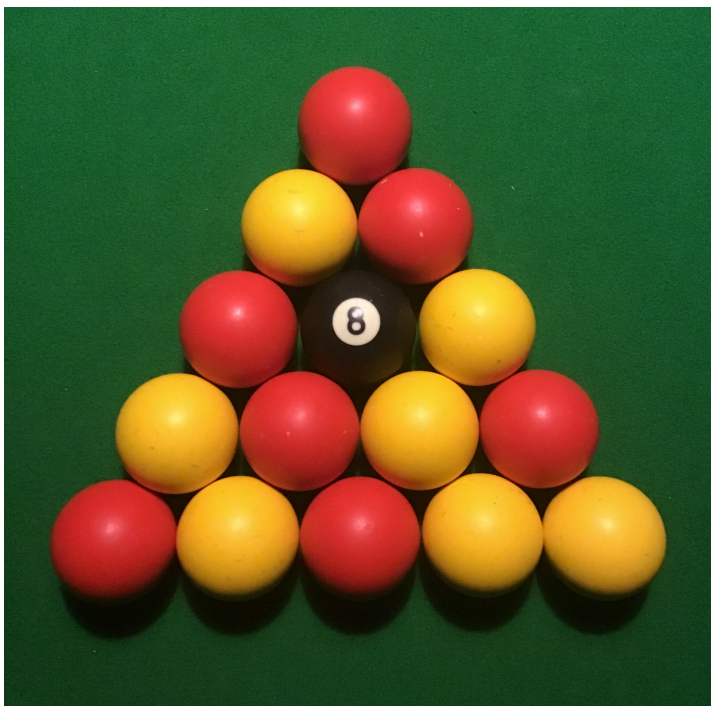
Perfectly lambertian surfaces don't really exist, but some real world surfaces come pretty close:

- Wood
- Unpolished rocks
- Frayed cloth

Basically, things that are rough and not shiny. These surfaces scatter light **diffusely**. Light that is scattered is **diffuse light**. Light that is reflected coherently without scattering is **specular light**.

Diffuse vs. specular light

These pool balls have specular highlights on them. They are smooth, not-very-diffuse surfaces. Image By SMcCandlish - Own work, CC BY-SA 4.0, link



These pool balls are a lot rougher. Their specular highlights are barely visible. They are more diffuse. Image By SouthcottC - Own work, CC BY-SA 4.0, link

Modelling diffuse light

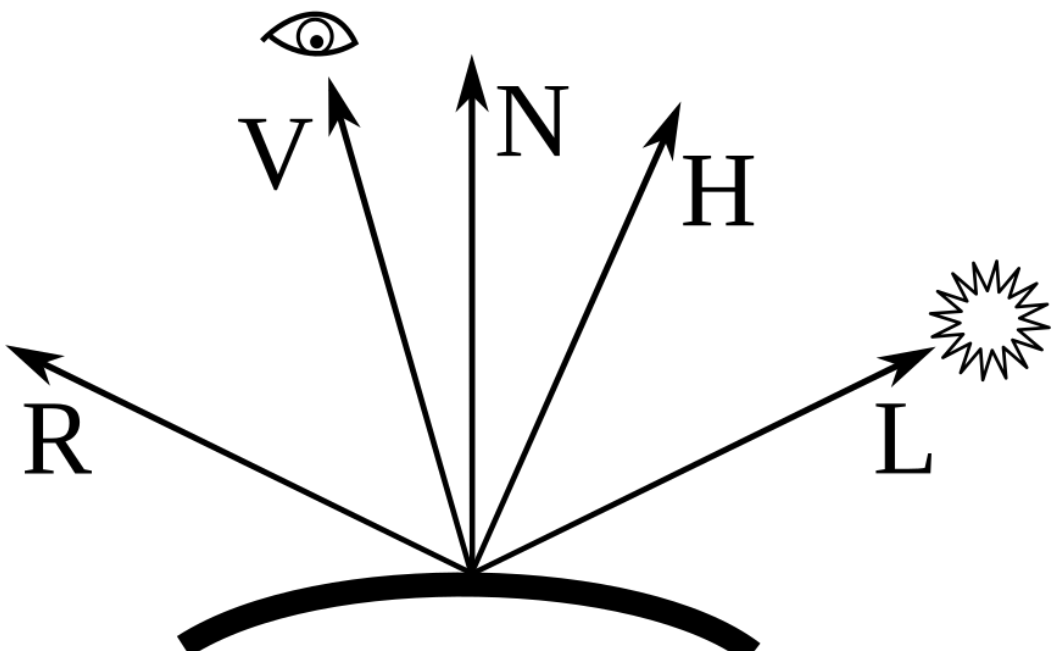
It turns out, when a surface is very diffuse, it is easy to calculate how bright it should be, and this is one of the things Lambert discovered.

Fundamentally, the amount of light the surface will receive is based on the angle between the surface's *normal* and the direction to the light.

The normal is the unit-length vector pointing exactly perpindicular away from the surface at a point.

And the direction to the light is a vector pointing from the point on the surface, in the direction of the light.

Modelling diffuse light (2)



Only consider `N` and `L` for now. `N` is the normal vector. `L` is the vector pointing to the light.

Modelling diffuse light (3)

It turns out, when the light is scattered in all directions, the brightness depends entirely on the cosine of the angle between `N` and `L`.

That is the surface is brightest when the surface is exactly pointing towards the light source, and gets dimmer as the surface rotates away, becoming 0 when it is perpendicular to or facing away from the light.

[How can we compute this quickly?]

Modelling diffuse light (4)

It's just the dot product:

```
let cos_angle = dot(normal, light_dir)
let diffuse_brightness = max(cos_angle, 0);
```

Remember: the dot product is just the magnitudes of the two vectors times the cosine between them.

In this case, the magnitudes are 1, so it ends up being just cosine.

Cosine is maximized at 1 when two vectors are exactly in line. That's when the surface scatters *all* the light, and will therefore be brightest. As it turns parallel, it scatters less and less, and gets dimmer.

Questions?

So where is the normal?

This calculation has to happen in one of the shaders, but that means the shader needs the normal.

Normals are normally something we associate with faces. A triangle, for example, has a normal defined by the cross product between two of its edges.

However, there is no “face shader” that we have access to in WebGPU. We need normals to be associated with vertices.

Normal attributes

So now it's finally time to add another attribute. Recall that attributes are pieces of information we associate with vertices.

Previously we saw:

- Position, which is basically required.
- Color
- UV coordinates

Now we are going to add the normal vector, representing a vector that is perpendicular to the surface at the vertex.

Computing the normal

In reality, if you build a 3D model in a program like Blender, it will compute and export the normals for you.

However, when normals are exported in OBJ files, they become annoying to process. Therefore, it's easier to compute them at runtime.

To compute normals, there is a simple algorithm that works well:

- For each vertex: store which faces it touches from the index list.
- For each face: compute the cross product.
- For each vertex again: add together all the cross products from each face it touches, and then normalize the result.

Computing the normal (2)

The logic behind this is that we want to approximate the normal at a point, but we only know the normal for sure in the regions around the point.

We don't want to treat them equally, otherwise we would end up with artifacts between lots of tiny triangles and large open faces.

Therefore, we weight them by area. This happens naturally as we take the cross product: the cross product is the product of the magnitudes of the two input vectors, so as the triangle gets larger the cross product is larger.

Computing the normal (3)

When we normalize the sum of cross products, we end up with the weighted average.

That is, suppose our vertex is touching 3 triangles: two with a cross product of length 1, and one with a cross product of length 4.

We take the sum of those three vertices. We get a vertex pointing roughly in between the three, but it's 6 units long. The length 4 one will contribute more to its total length in each dimension.

Therefore, when it gets normalized, it will still point more towards the direction of the big triangle.

Computing the normal code